

UDF - CENTRO UNIVERSITÁRIO

CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE

**RELATÓRIO DE ATIVIDADES
PRÁTICAS:
ARRAYS, MATRIZES E ALGORITMOS
DE
ORDENAÇÃO E BUSCA**

Avaliação Empírica de Complexidade e Estruturas de Dados Aplicadas (ED II)

Disciplina: Estrutura de Dados II
Professor: Karla Roberta
Estudantes: Pietro Vianna da Rocha e João Pedro Alves de Sousa
Sistemas Utilizados: Python 3.12, Matplotlib e ReportLab
Data da Entrega: Agosto / 2026

Brasília, DF
2026

1. INTRODUÇÃO E MODELO DE MEMÓRIA

Este relatório apresenta um estudo prático e teórico sobre estruturas de dados, algoritmos de ordenação e busca, relacionando arrays, matrizes, índices, loops e complexidade computacional. A comparação experimental considera a quantidade real de operações realizadas — comparações, trocas e movimentações —, evitando depender apenas do tempo de execução, que varia conforme o computador utilizado.

Em Python, listas são objetos mutáveis. Ao alterar um elemento de uma lista, a lista continua sendo o mesmo objeto e, portanto, mantém o mesmo valor retornado por `id()`. Quando um inteiro é substituído, a referência armazenada naquela posição passa a apontar para outro objeto, pois inteiros são imutáveis. Assim, `id()` deve ser entendido como a identidade do objeto, e não genericamente como um endereço físico de memória.

2. PESQUISA SOBRE ALGORITMOS DE ORDENAÇÃO

2.1. Bubble Sort (Ordenação por Bolha)

O Bubble Sort percorre repetidamente o vetor comparando pares adjacentes e realizando a troca quando o elemento da esquerda é maior que o da direita. Com uma flag de parada antecipada, o algoritmo pode encerrar após uma passagem sem trocas, detectando que o vetor já está ordenado.

Complexidade: melhor caso $O(n)$ com parada antecipada; caso médio $O(n^2)$; pior caso $O(n^2)$. É estável, in-place ($O(1)$ de memória auxiliar) e adequado principalmente a conjuntos pequenos, quase ordenados ou a fins didáticos.

2.2. Quick Sort (Ordenação Rápida)

O Quick Sort utiliza a estratégia de dividir para conquistar: escolhe um pivô, particiona o vetor em torno dele e ordena recursivamente as duas subpartições. No caso médio apresenta $O(n \log n)$, mas pode atingir $O(n^2)$ quando as partições ficam extremamente desbalanceadas.

Na implementação utilizada neste relatório, o último elemento é escolhido como pivô. Por isso, um vetor já ordenado pode provocar o pior caso $O(n^2)$; essa degradação não deve ser descrita simplesmente como “rara”, pois depende diretamente da estratégia de pivô e da distribuição dos dados.

2.3. Tabela Comparativa de Complexidade Teórica

Métrica	Bubble Sort	Quick Sort
Princípio	Comparação e troca de pares adjacentes	Dividir para conquistar; particionamento por pivô
Melhor caso	$O(n)$, com flag de parada	$O(n \log n)$
Caso médio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$, com partições muito desbalanceadas
Memória auxiliar	$O(1)$ - in-place	$O(\log n)$ em média - pilha recursiva
Estabilidade	Estável	Não estável na forma clássica
Indicação	Conjuntos pequenos/quase ordenados; didática	Conjuntos maiores e aplicações que exigem bom desempenho médio

Observação técnica

A complexidade $O(n \log n)$ do Quick Sort é substancialmente menor que $O(n^2)$ para grandes entradas, embora ambos possam produzir exatamente o mesmo vetor ordenado. Por isso, a análise da quantidade de operações é essencial para comparar eficiência, não apenas o resultado final.

3. IMPLEMENTAÇÃO E EXPERIMENTOS DE ORDENAÇÃO

3.1. Código Fonte - Bubble Sort (Python)

```
# Bubble Sort com contagem e parada antecipada
def bubble_sort(arr):
    a = arr[:]
    n = len(a)
    comparisons = swaps = 0
    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                trocou = True
        if not trocou:
            break
    return a, comparisons, swaps
```

3.2. Código Fonte - Quick Sort (Python)

```
# Quick Sort com último elemento como pivô
def quick_sort(arr):
    a = arr[:]
    comparisons = movements = 0
    def partition(lo, hi):
        nonlocal comparisons, movements
        pivot = a[hi]; i = lo - 1
        for j in range(lo, hi):
            comparisons += 1
            if a[j] <= pivot:
                i += 1; a[i], a[j] = a[j], a[i]; movements += 1
        a[i + 1], a[hi] = a[hi], a[i + 1]; movements += 1
        return i + 1
    def qs(lo, hi):
        if lo < hi:
            p = partition(lo, hi); qs(lo, p - 1); qs(p + 1, hi)
    qs(0, len(a) - 1)
    return a, comparisons, movements
```

3.3. Resultados Empíricos

Os testes utilizam os mesmos dados para os dois algoritmos, gerados com random.seed(42) e random.randint(0, 100000). Dessa forma, os valores abaixo são reproduíveis com o código adotado.

N	Bubble - Comparações	Bubble - Trocas	Quick - Comparações	Quick - Movimentações
10	45	26	29	15
20	175	87	65	41
1.000	498.324	243.438	10.218	6.231

3.4. Análise dos Resultados de Ordenação

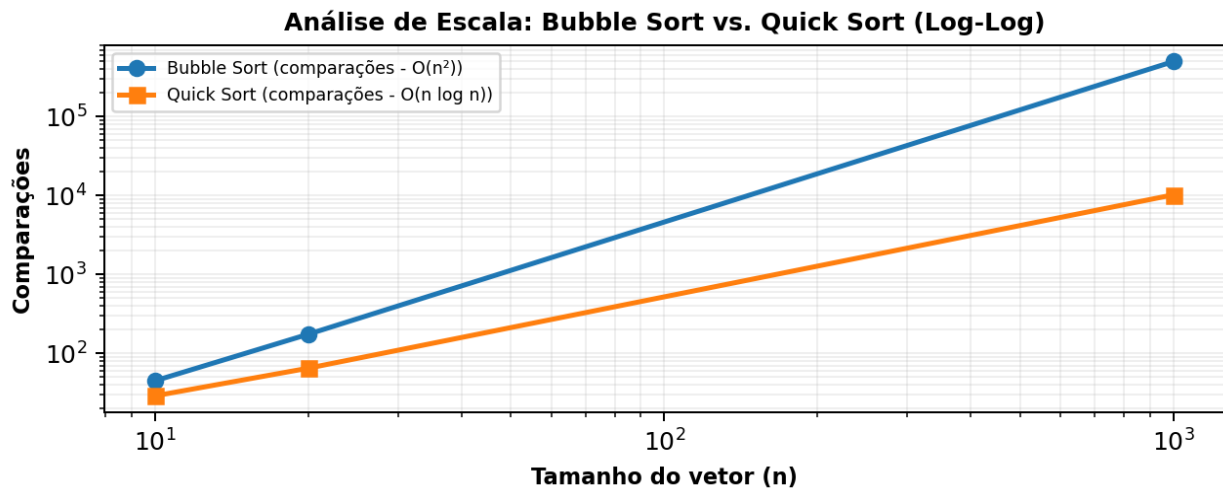


Figura 1: crescimento empírico das comparações em escala log-log.

a) Qual algoritmo realizou menos operações para 10 elementos?

O Quick Sort: 29 comparações e 15 movimentações, contra 45 comparações e 26 trocas do Bubble Sort.

b) O comportamento permaneceu igual para 20 elementos?

Sim. O Quick Sort continuou realizando menos operações, e a diferença aumentou conforme n cresceu.

c) O que ocorreu em $n = 1.000$?

A diferença tornou-se muito acentuada: 498.324 comparações no Bubble Sort contra 10.218 no Quick Sort, evidenciando $O(n^2)$ versus $O(n \log n)$.

d) Qual apresentou maior crescimento?

O Bubble Sort, pois seu crescimento quadrático aumenta muito mais rapidamente que $n \log n$.

e) Os resultados são coerentes com a teoria?

Sim. Os resultados empíricos reproduzem a tendência assintótica esperada.

f) Quando escolher Bubble Sort?

Em conjuntos pequenos, quase ordenados ou em situações didáticas em que simplicidade é mais importante que desempenho.

g) Quando escolher Quick Sort?

Em conjuntos maiores e aplicações que exigem bom desempenho médio, com atenção à estratégia de escolha do pivô.

4. INVESTIGAÇÃO DE BUSCA SEQUENCIAL EM MATRIZES

```
def busca_sequencial_matriz(matriz, valor):  
    comparacoes = 0  
    for i in range(len(matriz)):  
        for j in range(len(matriz[0])):  
            comparacoes += 1  
            if matriz[i][j] == valor:  
                return True, i, j, comparacoes  
    return False, -1, -1, comparacoes
```

4.2. Resultados Experimentais de Busca Sequencial

A busca foi analisada em três cenários: elemento no início, elemento na última posição e elemento inexistente. Se o alvo está efetivamente na última célula, todas as posições precisam ser comparadas; portanto, os totais corretos são 4, 100 e 10.000, respectivamente.

Dimensões	Nº total	Início	Última posição	Inexistente
2 × 2	4	1	4	4
10 × 10	100	1	100	100
100 × 100	10.000	1	10.000	10.000

4.3. Respostas às Perguntas Reflexivas

a) Encontrar um elemento no início exige menos operações porque a função retorna assim que a primeira correspondência é encontrada. b) Se o elemento não existe, todas as posições precisam ser verificadas. c) O pior caso ocorre no último elemento ou quando o valor é inexistente. d) Em uma matriz $m \times n$, a complexidade no pior caso é $O(m \times n)$. Em uma matriz quadrada $k \times k$, triplicar o lado multiplica o número de posições por 9.

5. RESOLUÇÃO DOS ESTUDOS DE CASO (HANDS-ON)

5.1. Hands On 1 - Análise do Vetor de Temperaturas

Para manter aderência ao modelo fornecido, foi utilizado o vetor: [19.0, 24.3, 21.3, 25.3, 25.6, 16.1, 15.2, 29.2, 19.4, 19.0].

Métrica Calculada	Valor Obtido
Média aritmética	21,44 °C
Maior temperatura	29,2 °C (índice 7)
Menor temperatura	15,2 °C (índice 6)
Valores acima da média	4
Percursos estimados	29 operações (10 + 9 + 10)
Complexidade total	$O(n)$ - linear

5.2. Hands On 2 - Monitoramento Dinâmico de Sensores Industriais

Foi considerada uma matriz 5×24 , em que cada linha representa um sensor e cada coluna representa um horário do dia. A análise percorre as 120 medições para calcular médias, localizar a maior temperatura e contabilizar leituras acima de um limite.

```
# sensores[i][j]: sensor i, hora j
medias_sensores = []
for i in range(5):
    soma = 0.0
    for j in range(24):
        soma += sensores[i][j]
    medias_sensores.append(soma / 24)

maior_valor = sensores[0][0]
sensor_maior = hora_maior = 0
for i in range(5):
    for j in range(24):
        if sensores[i][j] > maior_valor:
            maior_valor = sensores[i][j]
            sensor_maior, hora_maior = i, j
```

Sensor	Média das 24 medições
Sensor 0	24,69 °C
Sensor 1	25,09 °C
Sensor 2	24,31 °C
Sensor 3	24,02 °C
Sensor 4	25,48 °C

Grandeza	Valor obtido
Maior temperatura registrada	35,0 °C
Sensor responsável	Sensor 1
Horário da ocorrência	12h
Média geral (120 medições)	24,72 °C
Limite informado	28,0 °C
Leituras acima do limite	38

Por que loops aninhados? A matriz possui duas dimensões. O índice i identifica o sensor (linha) e j identifica o horário (coluna). Assim, `sensores[i][j]` localiza uma medição específica. Uma varredura completa visita $5 \times 24 = 120$ posições, caracterizando $O(m \times n)$.

6. ANÁLISE COMPILADA E DISCUSSÃO

Os experimentos mostram que o aumento da entrada influencia diretamente o número de operações, mas a taxa de crescimento depende do algoritmo. O Bubble Sort apresenta crescimento quadrático $O(n^2)$, enquanto o Quick Sort apresenta $O(n \log n)$ no caso médio. Em $n = 1.000$, o Bubble Sort realizou 498.324 comparações e o Quick Sort 10.218 — aproximadamente 48,8 vezes menos comparações para o Quick Sort.

Por isso, não é suficiente observar apenas o vetor final ordenado: dois algoritmos podem produzir exatamente o mesmo resultado e, ainda assim, executar quantidades muito diferentes de operações. A análise de comparações, trocas, movimentações e percursos evidencia a eficiência computacional de cada abordagem.

7. CONCLUSÃO

Este trabalho consolidou conceitos fundamentais de arrays, matrizes, ordenação, busca e complexidade computacional. Vetores são adequados a dados unidimensionais, enquanto matrizes representam naturalmente informações organizadas por duas dimensões. Em percursos simples, o custo cresce linearmente com o número de elementos; em matrizes, a varredura completa é $O(m \times n)$.

Na ordenação, a escolha do algoritmo torna-se especialmente relevante conforme o volume de dados cresce. O Bubble Sort, embora simples e útil didaticamente, torna-se pouco escalável devido ao comportamento $O(n^2)$. O Quick Sort, por sua vez, apresenta crescimento $O(n \log n)$ no caso médio, sendo muito mais eficiente para entradas maiores, embora possa degradar para $O(n^2)$ com particionamentos muito desbalanceados.

8. REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H. et al. Introduction to Algorithms. 4th ed. MIT Press, 2022.

KNUTH, Donald E. The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd ed. Addison-Wesley, 1998.

Nota de consistência: os resultados de ordenação foram recalculados com o mesmo código e a mesma semente do modelo fornecido. Na busca em matriz, manteve-se a contagem matematicamente consistente para um elemento localizado na última posição: $m \times n$ comparações.